

Leave-one-out disturbance ablation

Train on everything, then drop one disturbance at a time and see what the success rate does.

What was run

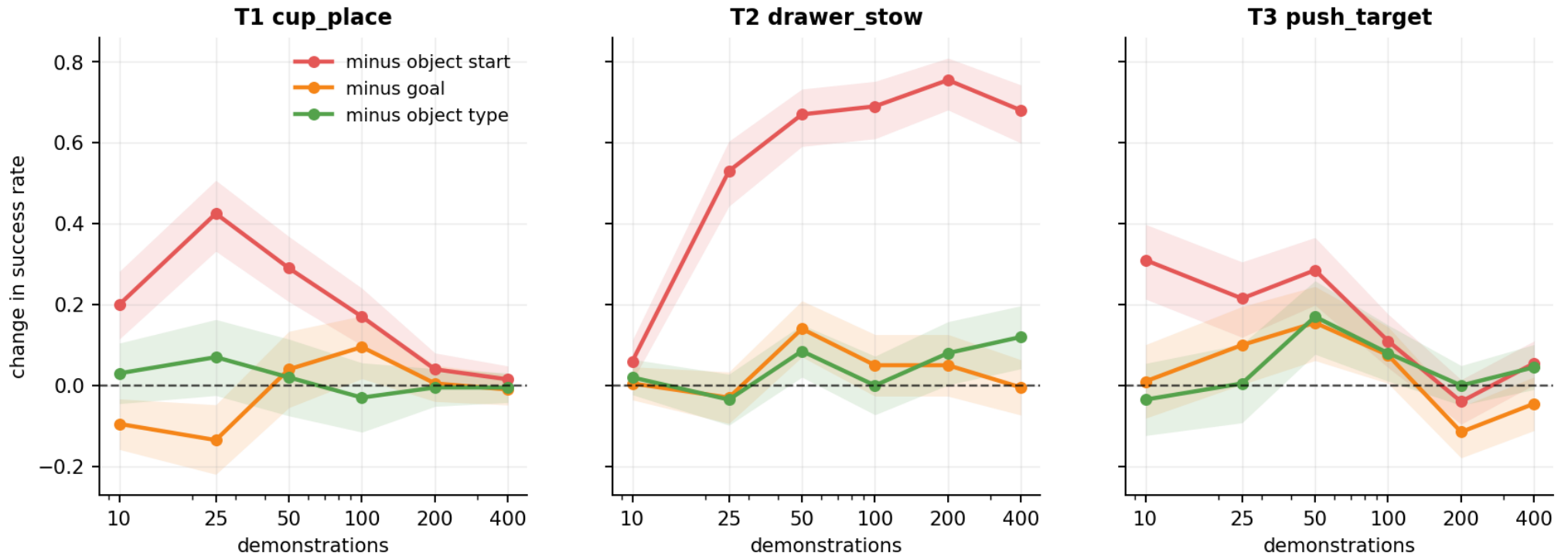
Four setups per task. Every one is trained at six demonstration budgets (10, 25, 50, 100, 200, 400) and scored on 200 episodes, so $4 \times 6 \times 3$ tasks = 72 policies and 14,400 episodes. Same training recipe everywhere, one seed, 80,000 steps.

setup	object start position	goal position	object type	name on disk
all on	varies	varies	10 types	L3b
minus object start	FIXED	varies	10 types	BC
minus goal	varies	FIXED	10 types	AC
minus object type	varies	varies	1 type	L2

The three tasks are T1 cup_place (put a cup on a goal disk), T2 drawer_stow (open a drawer and stow an object in it) and T3 push_target (push a puck to a target). The change reported for a disturbance is always **success rate without it, minus success rate with all three on, at the same demonstration budget**. Positive means the simpler setup did better.

What dropping each disturbance does

What dropping one disturbance does, against keeping all three



Change in success rate against keeping all three disturbances. Bands are 95 % intervals. Above the dashed line the simpler setup did better.

T1 cup_place

demonstrations	minus object start	minus goal	minus object type
10	+0.200 [+0.115, +0.282]	-0.095 [-0.158, -0.033]	+0.030 [-0.045, +0.105] (not clear of zero)
25	+0.425 [+0.332, +0.507]	-0.135 [-0.220, -0.048]	+0.070 [-0.025, +0.163] (not clear of zero)
50	+0.290 [+0.207, +0.368]	+0.040 [-0.055, +0.134] (not clear of zero)	+0.020 [-0.075, +0.115] (not clear of zero)
100	+0.170 [+0.099, +0.240]	+0.095 [+0.017, +0.172]	-0.030 [-0.115, +0.056] (not clear of zero)
200	+0.040 [+0.006, +0.080]	+0.005 [-0.039, +0.050] (not clear of zero)	-0.005 [-0.052, +0.041] (not clear of zero)
400	+0.015 [-0.014, +0.048] (not clear of zero)	-0.010 [-0.048, +0.027] (not clear of zero)	-0.005 [-0.042, +0.031] (not clear of zero)

T2 drawer_stow

demonstrations	minus object start	minus goal	minus object type
10	+0.060 [+0.011, +0.112]	+0.005 [-0.036, +0.046] (not clear of zero)	+0.020 [-0.023, +0.065] (not clear of zero)
25	+0.530 [+0.443, +0.604]	-0.030 [-0.094, +0.033] (not clear of zero)	-0.035 [-0.098, +0.028] (not clear of zero)
50	+0.670 [+0.591, +0.733]	+0.140 [+0.071, +0.209]	+0.085 [+0.021, +0.150]
100	+0.690 [+0.610, +0.752]	+0.050 [-0.026, +0.126] (not clear of zero)	+0.000 [-0.072, +0.072] (not clear of zero)
200	+0.755 [+0.681, +0.809]	+0.050 [-0.026, +0.126] (not clear of zero)	+0.080 [+0.001, +0.158]
400	+0.680 [+0.599, +0.743]	-0.005 [-0.073, +0.063] (not clear of zero)	+0.120 [+0.042, +0.197]

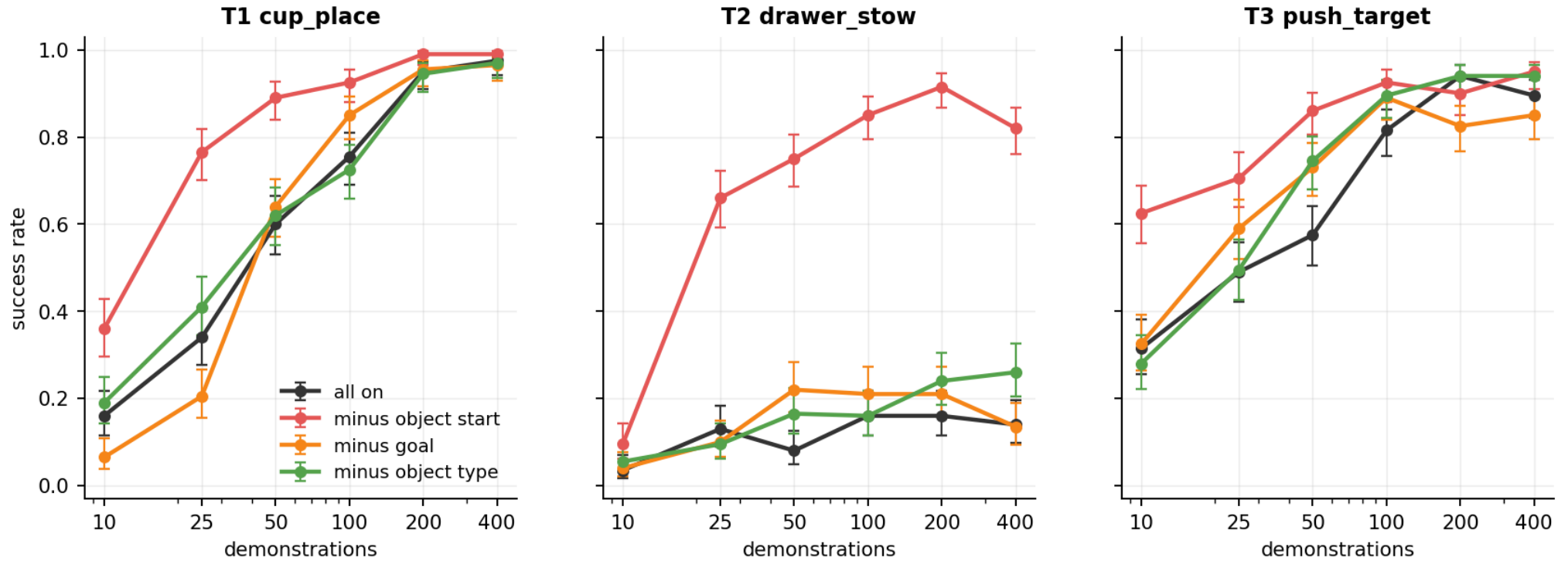
T3 push_target

demonstrations	minus object start	minus goal	minus object type
10	+0.310 [+0.214, +0.398]	+0.010 [-0.081, +0.101] (not clear of zero)	-0.035 [-0.124, +0.054] (not clear of zero)
25	+0.215 [+0.119, +0.305]	+0.100 [+0.002, +0.195]	+0.005 [-0.092, +0.102] (not clear of zero)
50	+0.285 [+0.199, +0.366]	+0.155 [+0.062, +0.245]	+0.170 [+0.077, +0.259]
100	+0.110 [+0.044, +0.176]	+0.075 [+0.005, +0.145]	+0.080 [+0.011, +0.149]
200	-0.040 [-0.096, +0.014] (not clear of zero)	-0.115 [-0.179, -0.052]	+0.000 [-0.049, +0.049] (not clear of zero)
400	+0.055 [+0.002, +0.110]	-0.045 [-0.111, +0.021] (not clear of zero)	+0.045 [-0.010, +0.101] (not clear of zero)

Read each budget on its own line. The same disturbance can matter at 25 demonstrations and not at 400.

The success rates behind those changes

Success rate of each setup



Success rate of each setup. Error bars are 95 % intervals on 200 episodes.

T1 cup_place

demonstrations	all on	minus object start	minus goal	minus object type
10	0.160 (32/200)	0.360 (72/200)	0.065 (13/200)	0.190 (38/200)
25	0.340 (68/200)	0.765 (153/200)	0.205 (41/200)	0.410 (82/200)
50	0.600 (120/200)	0.890 (178/200)	0.640 (128/200)	0.620 (124/200)
100	0.755 (151/200)	0.925 (185/200)	0.850 (170/200)	0.725 (145/200)
200	0.950 (190/200)	0.990 (198/200)	0.955 (191/200)	0.945 (189/200)
400	0.975 (195/200)	0.990 (198/200)	0.965 (193/200)	0.970 (194/200)

T2 drawer_stow

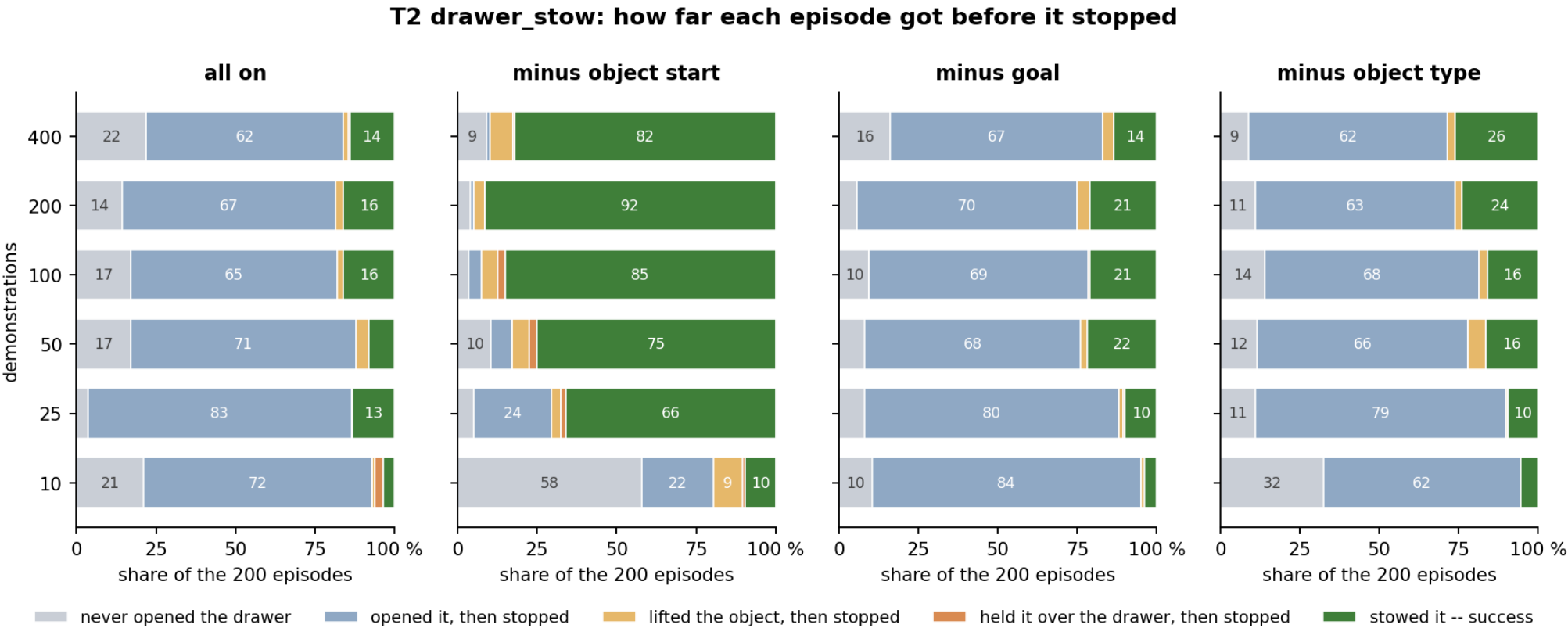
demonstrations	all on	minus object start	minus goal	minus object type
10	0.035 (7/200)	0.095 (19/200)	0.040 (8/200)	0.055 (11/200)
25	0.130 (26/200)	0.660 (132/200)	0.100 (20/200)	0.095 (19/200)
50	0.080 (16/200)	0.750 (150/200)	0.220 (44/200)	0.165 (33/200)
100	0.160 (32/200)	0.850 (170/200)	0.210 (42/200)	0.160 (32/200)
200	0.160 (32/200)	0.915 (183/200)	0.210 (42/200)	0.240 (48/200)
400	0.140 (28/200)	0.820 (164/200)	0.135 (27/200)	0.260 (52/200)

T3 push_target

demonstrations	all on	minus object start	minus goal	minus object type
10	0.315 (63/200)	0.625 (125/200)	0.325 (65/200)	0.280 (56/200)
25	0.490 (98/200)	0.705 (141/200)	0.590 (118/200)	0.495 (99/200)
50	0.575 (115/200)	0.860 (172/200)	0.730 (146/200)	0.745 (149/200)
100	0.815 (163/200)	0.925 (185/200)	0.890 (178/200)	0.895 (179/200)
200	0.940 (188/200)	0.900 (180/200)	0.825 (165/200)	0.940 (188/200)
400	0.895 (179/200)	0.950 (190/200)	0.850 (170/200)	0.940 (188/200)

Where T2 policies got stuck

T2 is a four-part task: open the drawer, pick up the object, carry it over the open drawer, drop it in. Every episode records how far it got, so a failure can be placed instead of just counted. Each bar below is 200 episodes split by the furthest point reached, so the five shares add to 100.



How far each episode got before it stopped. Numbers are percentages of 200 episodes.

setup	demos	never opened	opened, stopped	lifted, stopped	held over, stopped	stowed it
all on	10	42	144	2	5	7
all on	25	7	166	1	0	26
all on	50	34	142	8	0	16
all on	100	34	130	4	0	32
all on	200	29	134	5	0	32
all on	400	44	124	3	1	28

setup	demos	never opened	opened, stopped	lifted, stopped	held over, stopped	stowed it
minus object start	10	116	45	18	2	19
minus object start	25	10	49	6	3	132
minus object start	50	21	13	11	5	150
minus object start	100	7	8	10	5	170
minus object start	200	8	2	7	0	183
minus object start	400	18	2	15	1	164
minus goal	10	21	169	2	0	8
minus goal	25	16	160	3	1	20
minus goal	50	16	136	4	0	44
minus goal	100	19	138	0	1	42
minus goal	200	11	139	8	0	42
minus goal	400	32	134	7	0	27
minus object type	10	65	124	0	0	11
minus object type	25	22	158	1	0	19
minus object type	50	23	133	11	0	33
minus object type	100	28	135	5	0	32
minus object type	200	22	126	4	0	48
minus object type	400	18	125	5	0	52

Counts out of 200 episodes.

What could be wrong with these numbers

#	issue	what was measured
1	The two "minus" setups that need new demonstrations were generated on different GPUs.	Demonstrations for minus-object-start and minus-goal came off A100s; the other two setups' demonstrations came off an RTX 4090. A control policy trained on A100-generated data scored 0.750 where its 4090-generated twin scored 0.860 (difference -0.110, 95 % CI -0.208 to -0.012). That control changed the GPU and the random seed together, so it bounds the effect rather than pinning it down. Any change in this report smaller than about 0.11 sits inside that boundary.
2	One training seed.	Every policy is seed 0. The intervals here cover episode sampling only; they contain no training-run variance.
3	Repeating an evaluation does not give the identical episodes back.	Re-running one 20-episode slice with the same seed and checkpoint reproduced the slice total but flipped two individual episodes. Expect about +/-1 success per 20 episodes of jitter.
4	"Minus object type" is not a newly generated setup.	Removing the object axis from the full set gives exactly the existing single-object setup, verified identical over 300 matched resets. It is re-used rather than regenerated, which is also why it is the only comparison whose two sides see the same starting positions.
5	Do not average a change across demonstration budgets.	The six budgets are six different policies. A test for a shared success rate is rejected in all 18 setup-task pairs, so an average over budgets describes the mix of budgets chosen, not the setup.

What would fix them

#	next step
1	Re-run the GPU control with the demonstration seed held fixed, so the GPU is the only thing that changes.
2	Train more than one seed on whichever comparisons matter.
3	Record the evaluator version in the results ledger, and which git revision each run used.
4	Measure whether a second warm-up batch moves the scored success rate.
5	Guard the pooling script against mixing evaluator versions, and assert the termination-term ordering the success signal depends on.

Every number here comes from `src/cog/analysis/loo.py`, over the 108 pooled result files in `results/`. The long edition, with all six arms of the sweep, the per-slice dispersion, timing distributions and the full flaw list, is `paper/loo_report/loo_report_full.pdf`. The tables are also plain CSV in `paper/loo_report/tables/`.